

MODULE 7

DevOps and CI/CD for Microservices

From source-code change to a secure, tested production deployment.



LEARNING OBJECTIVES

Git workflows, CI/CD, testing, GitHub/Jenkins, deployment strategies and security.

BY THE END OF THIS MODULE, YOU CAN

1. Explain the DevOps lifecycle
2. Apply Git workflows
3. Explain CI, continuous delivery and deployment
4. Design a CI/CD pipeline
5. Understand GitHub Actions and Jenkins
6. Automate software testing
7. Build and promote immutable artifacts
8. Explain Blue/Green and Canary deployment
9. Understand authentication and authorization
10. Integrate security checks into CI/CD

FOUR AREAS

1 Source control & collaboration



2 CI/CD



3 Release strategies



4 Security



KEY TAKEAWAY Make delivery repeatable, automated, testable, secure and observable.

FROM MODULE 6 TO MODULE 7

Our services are resilient and observable — now deliver every change safely.

MODULES 1–6 GAVE US

- ✓ Well-bounded Spring Boot services
- ✓ REST and event contracts
- ✓ Service-owned data and Sagas
- ✓ Resilience patterns and failure tests
- ✓ Logs, metrics, traces and SLOs

NOW Automate how every change is reviewed, tested, secured and released.



OUR PATH THROUGH MODULE 7

- 1 DevOps culture and the delivery lifecycle
- 2 Git workflows and pull-request gates
- 3 CI, continuous delivery and pipelines
- 4 GitHub Actions, Jenkins and testing
- 5 Build once, promote the same image
- 6 Blue/Green and canary releases
- 7 Security in the pipeline

Part 1

Part 1

Part 2

Part 2

Part 2

Part 3

Part 4

KEY TAKEAWAY Modules 1–6 built good services; Module 7 makes delivering them repeatable, automated and safe.

PART 1: SOURCE CONTROL AND COLLABORATION

Everything we deliver starts as a reviewed change in Git.



SO FAR

- ✓ Services deploy independently
- ✓ Tests prove resilience and fallbacks
- ✓ Observability shows how releases behave

NEXT QUESTION

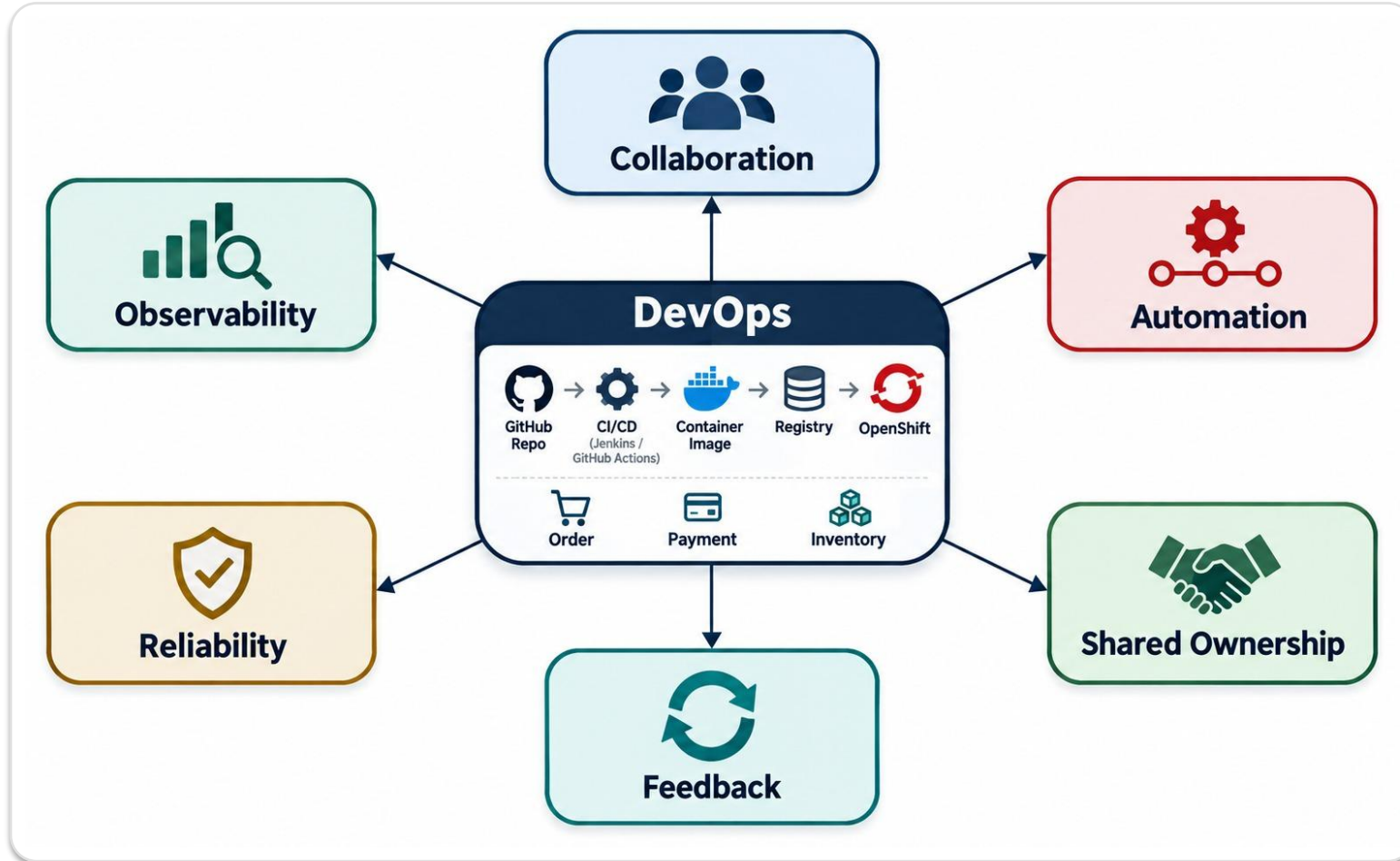
How do many teams change many services every day without breaking each other?

IN THIS PART

- 1 What DevOps means for microservices
- 2 The DevOps and delivery lifecycles
- 3 Git as the source of truth
- 4 Team workflows and branching models
- 5 Exercise: design a branching workflow
- 6 Pull requests and quality gates

WHAT DEVOPS MEANS FOR MICROSERVICES

Collaboration, automation, ownership, rapid feedback and reliable releases.



WHAT DEVOPS IS NOT

- ✗ Hand code over the wall to operations
- ✗ Buy a tool and call it DevOps
- ✗ Release once a quarter

You build it, you run it

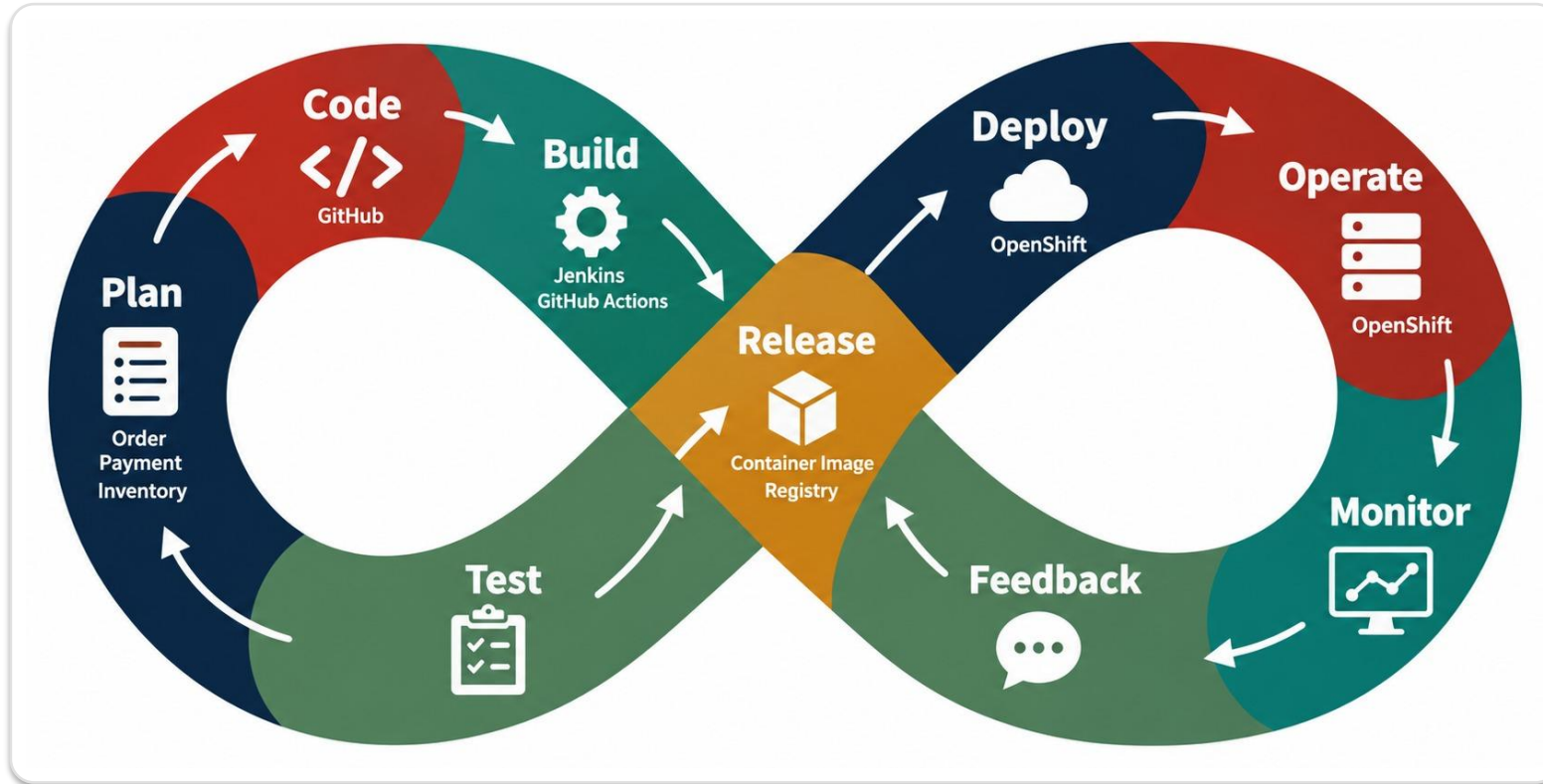
The team that owns Order Service also owns its pipeline, alerts and releases.

WHY IT MATTERS Dozens of services deploying independently need automation.

KEY TAKEAWAY DevOps is culture, practices and automation — essential when many services deploy independently.

THE DEVOPS LIFECYCLE

Plan → Code → Build → Test → Release → Deploy → Operate → Monitor → Feedback.



Left loop — deliver

Plan, code, build and test turn an idea into a validated change.

Right loop — run

Release, deploy, operate and monitor keep it healthy in production.

FEEDBACK THAT DRIVES PLANNING

Error rate

Latency

Incidents

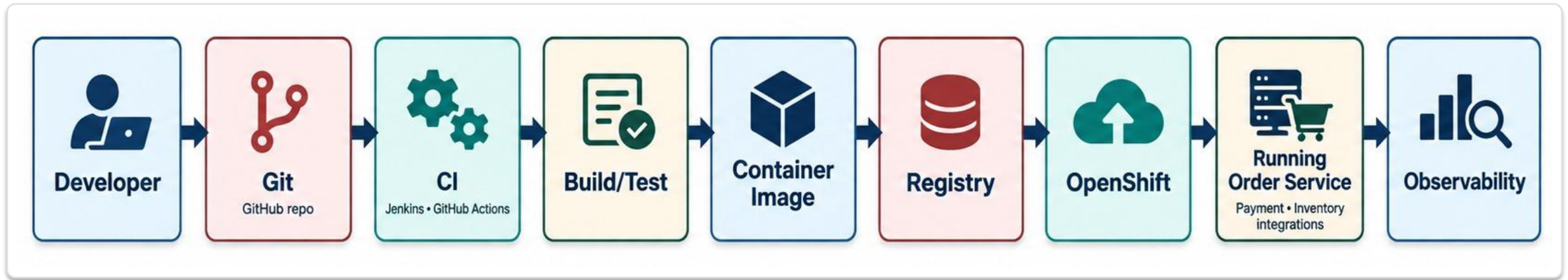
User feedback

A LOOP not a one-way pipeline.

KEY TAKEAWAY DevOps is a loop, not a one-way pipeline.

MICROSERVICES DELIVERY LIFECYCLE

Code change → Git → CI → container → registry → deployment → monitoring.



WHAT CI RUNS FOR ORDER SERVICE

- ✓ Compile and unit test
- ✓ Integration and contract tests
- ✓ Security scans
- ✓ Build and tag the image

After deployment

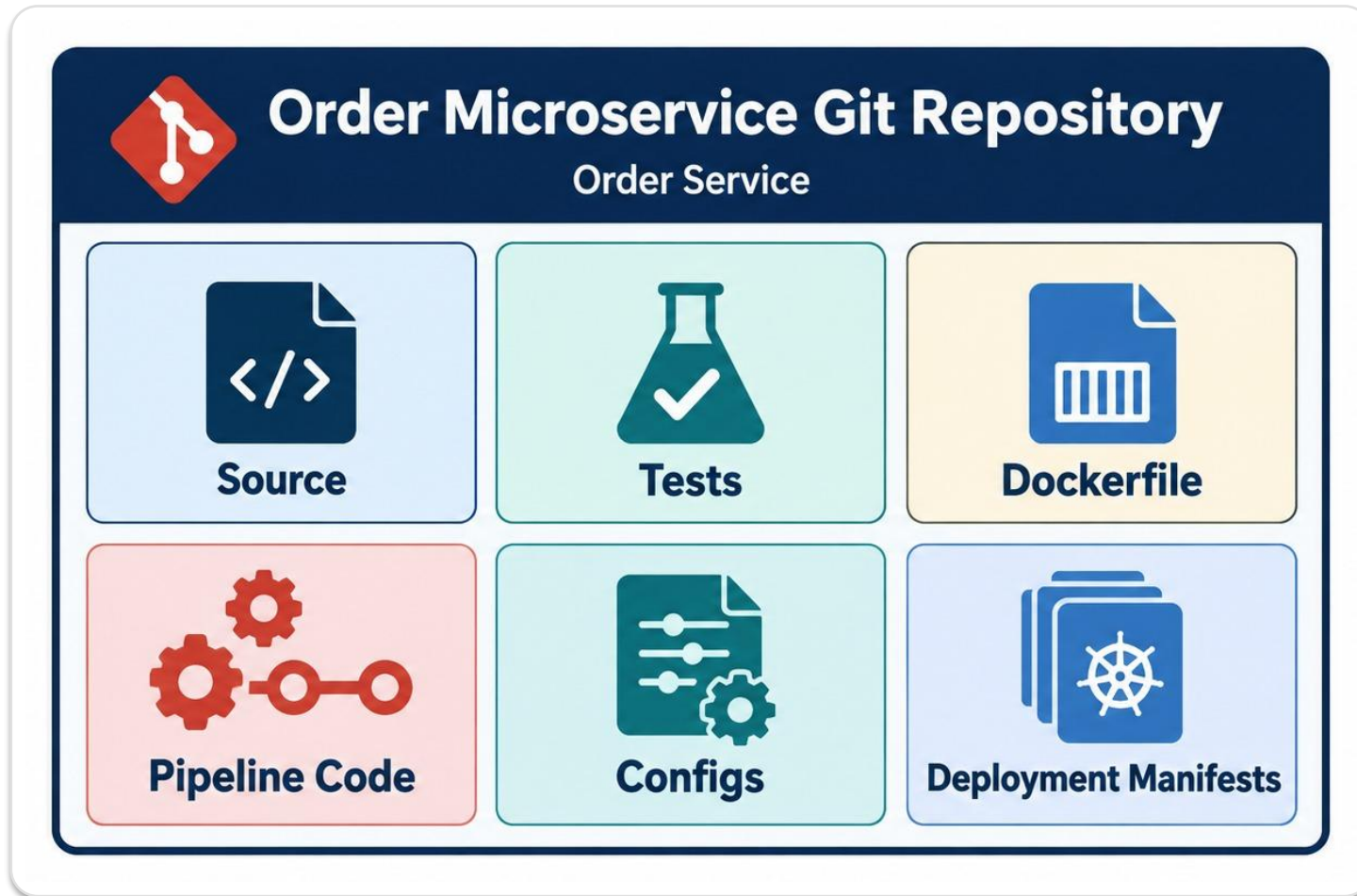
Readiness, smoke tests and dashboards confirm the release is healthy.

PER SERVICE Each service has its own pipeline and release cadence.

KEY TAKEAWAY Every Order Service change follows the same automated path from commit to feedback.

GIT AS THE SOURCE OF TRUTH

Source code, configuration, pipeline definitions, infrastructure and history.



One repository, whole recipe

```
order-service/  
├── src/main/java/...  
├── src/test/java/...  
├── Containerfile  
├── .github/workflows/ci.yml  
├── Jenkinsfile  
└── deploy/openshift/*.yaml
```

WHAT YOU GET

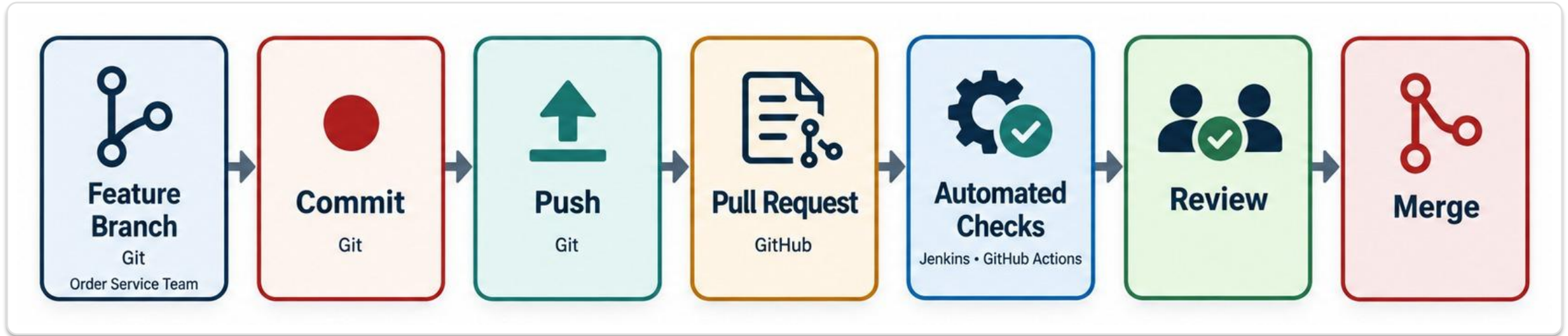
- ✓ Review
- ✓ Audit history
- ✓ Rollback
- ✓ Reproducible builds

NEVER IN GIT passwords, tokens or keys.

KEY TAKEAWAY Git holds the code, tests, pipeline and deployment definitions — the whole delivery recipe.

GIT WORKFLOW FOR A MICROSERVICE TEAM

Branch → commit → push → pull request → review → merge.



An example change

```
git switch -c feature/payment-limit-rule
git commit -m "feat: reject orders above limit"
git push -u origin feature/payment-limit-rule
# open a PR against main
```

Branch names

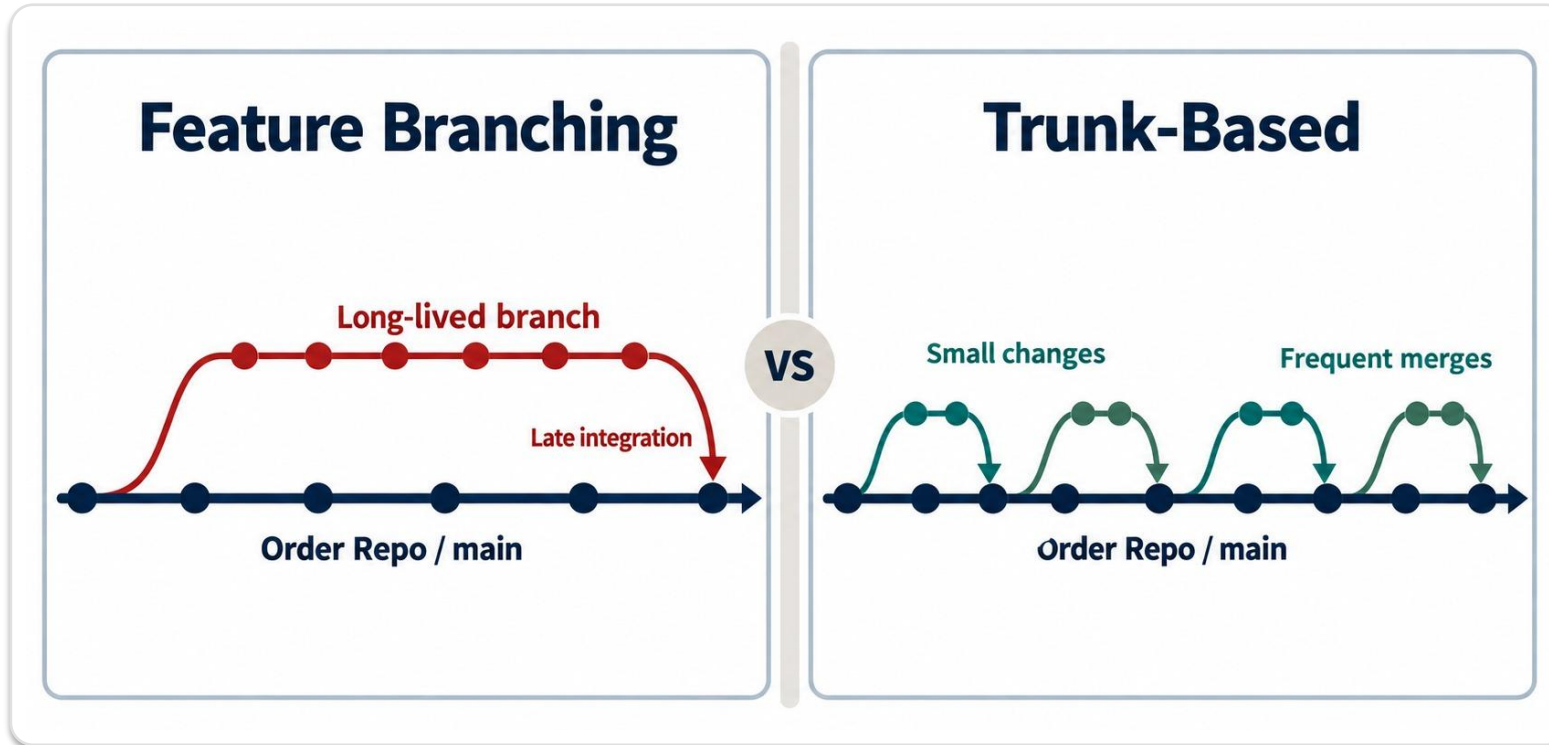
feature/..., bugfix/..., hotfix/... — link each change to a work item.

RULE Main is always releasable.

KEY TAKEAWAY Branch, commit, push, open a PR, pass checks and review — then merge.

TRUNK-BASED DEVELOPMENT VS. FEATURE BRANCHING

Two common Git collaboration models and their trade-offs.



Feature branching

Isolated work, but long-lived branches drift and merges get painful.

Trunk-based

Small changes merged at least daily; feature flags hide unfinished work.

REAL LESSON Integrate small batches often, whichever model you pick.

KEY TAKEAWAY Choose a workflow that supports frequent, safe integration — not large unvalidated batches.

EXERCISE: DESIGN A BRANCHING WORKFLOW

Agree how the Order Service team moves changes to production.

PRACTICE EXERCISE

10 MIN

SCENARIO

Six developers own Order Service. They release several times a week and must be able to ship an urgent fix within an hour.

Today some branches live for three weeks and merges break main.

YOUR TASKS

- 1 Choose trunk-based or short-lived feature branches — and justify it
- 2 Name the branch types and naming rules
- 3 List what must pass before a PR can merge
- 4 Describe the path for an urgent production fix

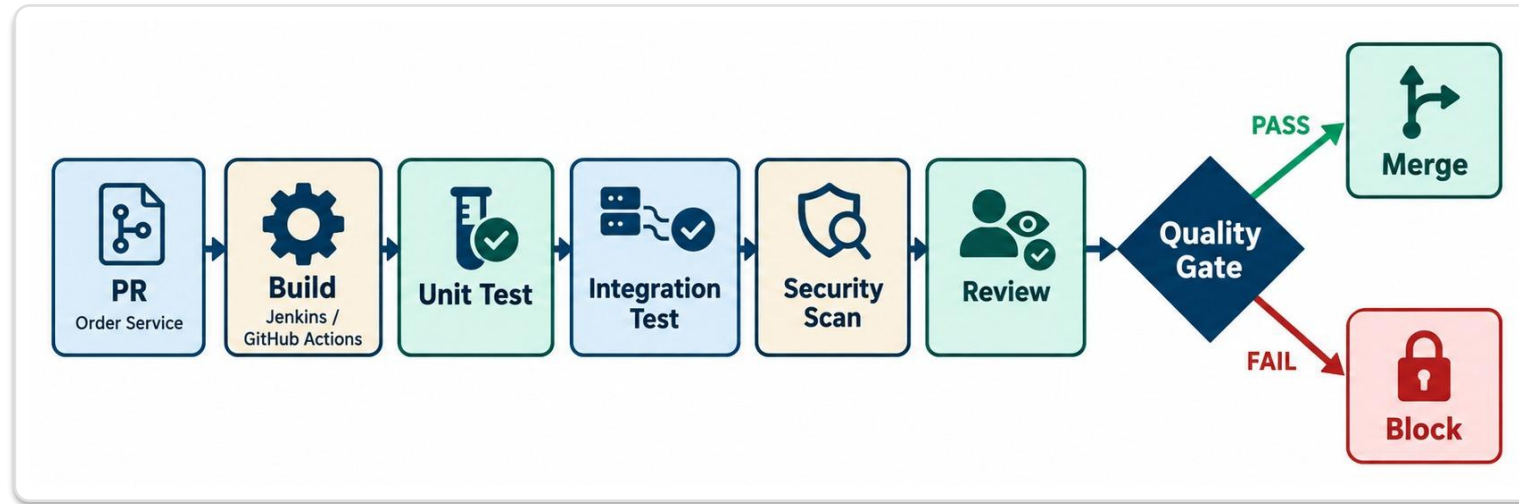
EXPECTED OUTCOME

- ✓ Short-lived branches merged within 1–2 days
- ✓ feature/, bugfix/, hotfix/ with a work-item link
- ✓ Build, tests, scan and one approval to merge
- ✓ Hotfix branch from main, same gates, fast review
- ✓ Feature flags for unfinished work

KEY TAKEAWAY A good workflow keeps branches short, gates every merge and still allows a fast, safe hotfix.

PULL REQUESTS AND AUTOMATED QUALITY GATES

Reviews, tests, linting, security checks and merge controls.



BRANCH PROTECTION ON MAIN

- ✓ At least one approval
- ✓ All required checks green
- ✓ No CRITICAL vulnerabilities
- ✓ Branch up to date

When a check fails

The merge button stays disabled; the author fixes and pushes again.

FAST FEEDBACK Keep PR checks under about ten minutes.

KEY TAKEAWAY Pull requests gate quality before code reaches main — fast feedback is cheap feedback.

PART 2: CI/CD PIPELINES

Turn every merge into a tested, versioned, deployable artifact.



SO FAR

- ✓ DevOps is a feedback loop
- ✓ Git holds the whole delivery recipe
- ✓ Pull requests gate every change

NEXT QUESTION

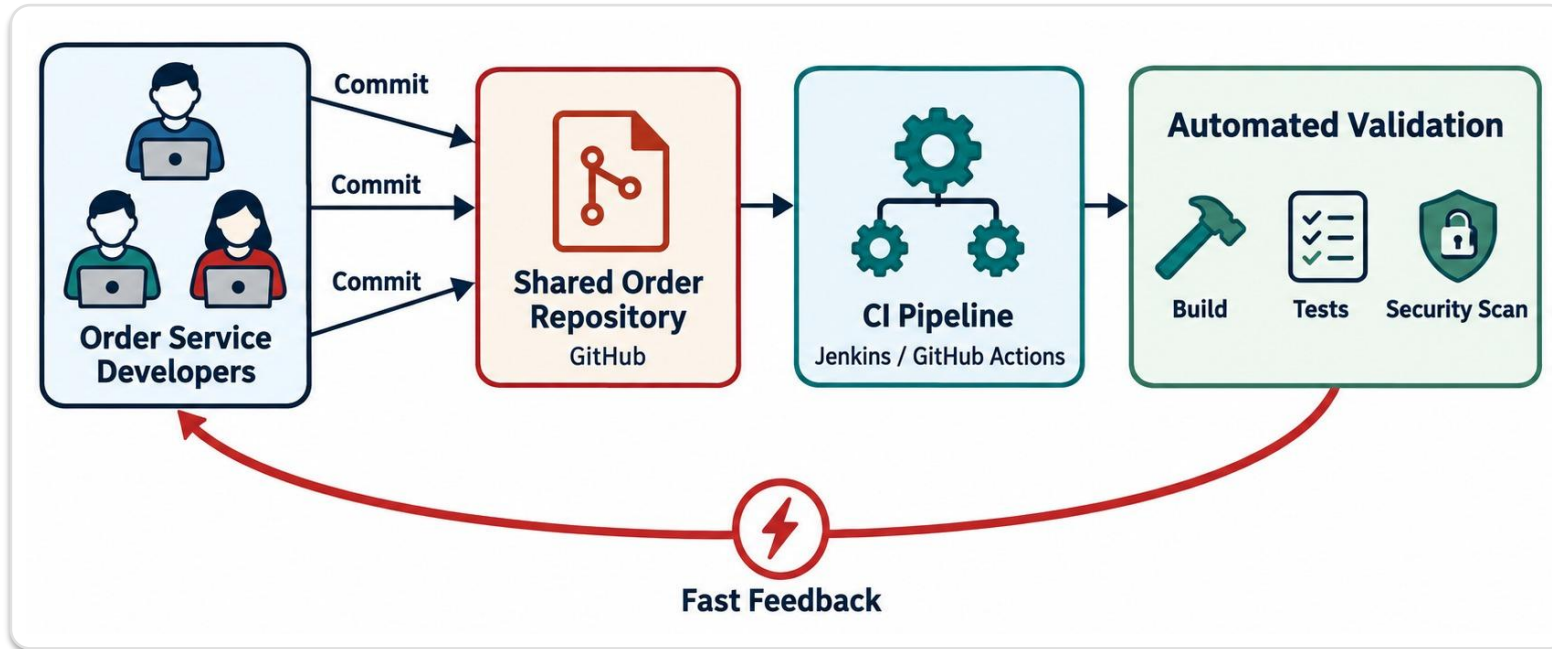
What should happen automatically between a merge and a running release?

IN THIS PART

- 1 Continuous integration, delivery and deployment
- 2 Pipeline architecture
- 3 GitHub Actions and Jenkins — in code
- 4 Automated testing across the pipeline
- 5 Exercise 3.3: security and resilience tests
- 6 Build once, promote — and Exercise 4.3

WHAT IS CONTINUOUS INTEGRATION?

Frequent integration with automated build and test feedback.



The classic problem

Weeks of separate work, then an "integration phase" full of surprises.

THE CI HABIT

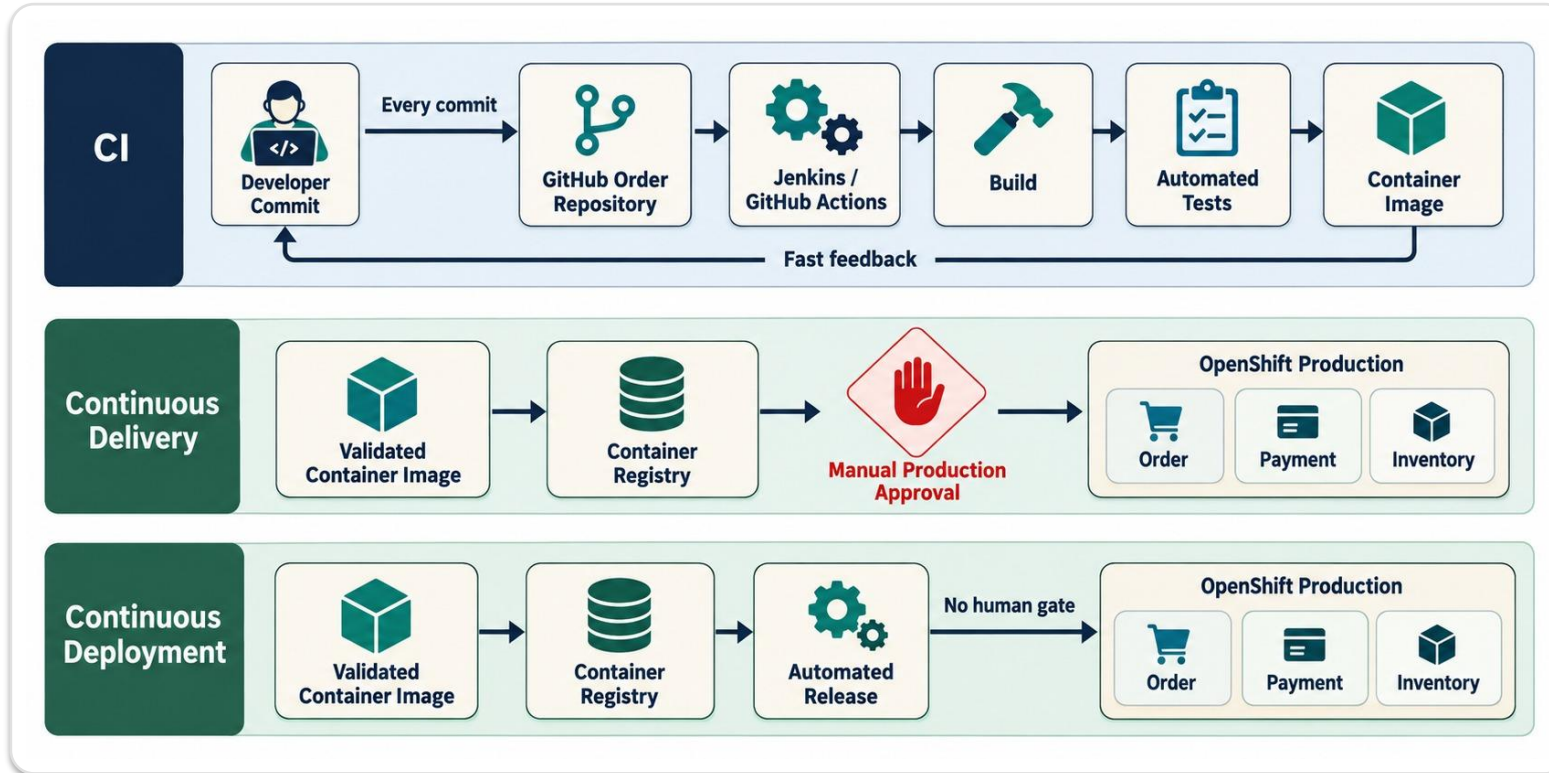
- ✓ Merge small changes daily
- ✓ Every push builds and tests
- ✓ Fix a broken build first

A PRACTICE not just "we have Jenkins."

KEY TAKEAWAY CI is a practice of frequent, validated integration — not just "we have Jenkins."

WHAT IS CONTINUOUS DELIVERY AND DEPLOYMENT?

CI vs. continuous delivery vs. continuous deployment.



Continuous delivery

Every change is releasable; a person approves the production step.

Continuous deployment

Every change that passes all gates goes to production automatically.

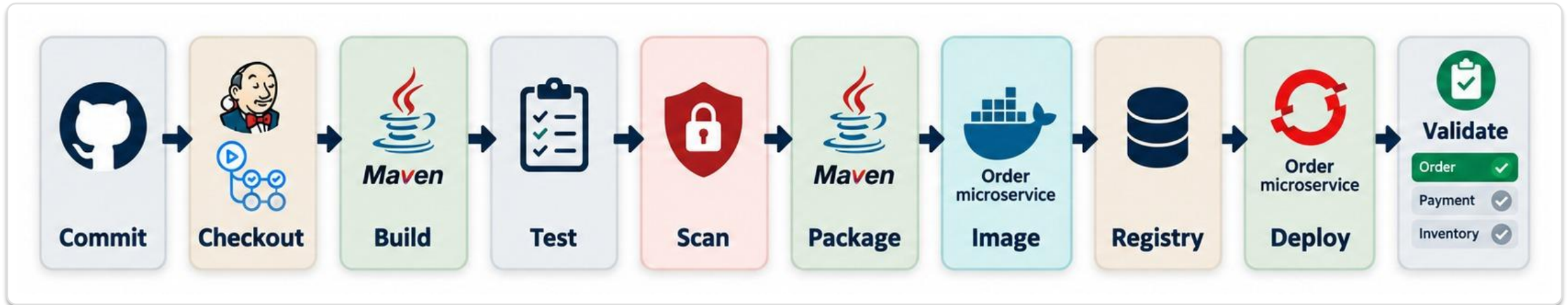
THREE QUESTIONS

- ? Can we release any time?
- ? Who approves production?
- ? What evidence do we need?

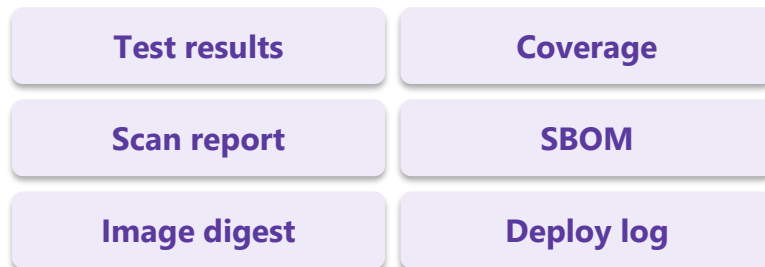
KEY TAKEAWAY Delivery keeps software releasable; deployment releases validated changes automatically.

CI/CD PIPELINE ARCHITECTURE

Commit → build → test → scan → package → publish → deploy.



EVIDENCE EACH RUN LEAVES



Stop conditions

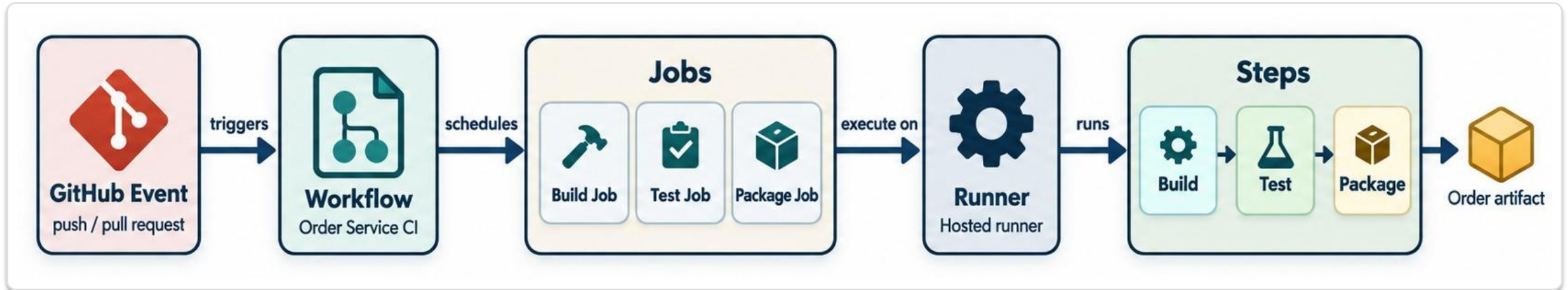
Failed tests, CRITICAL vulnerabilities or failed smoke checks end the run.

BACKBONE Every other slide in this module plugs into this.

KEY TAKEAWAY This pipeline is the backbone for the rest of Module 7.

GITHUB ACTIONS FOR MICROSERVICES CI/CD

Workflow triggers, jobs, runners and build/test/package stages.



Workflow

A YAML file in `.github/workflows`, versioned with the code.

Triggers

`pull_request`, `push`, `tags` or a manual `workflow_dispatch`.

Runners

GitHub-hosted or self-hosted inside the bank's network.

KEY IDEA The pipeline lives next to the code it builds.

KEY TAKEAWAY A Git event triggers automatic validation — defined right next to the code.

A GITHUB ACTIONS WORKFLOW FOR ORDER SERVICE

Build and test every PR; on main, build, push and tag the image with the commit.

✓ .github/workflows/ci.yml

```
name: order-service-ci
on:
  pull_request: { branches: [ main ] }
  push: { branches: [ main ] }
jobs:
  build-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: 21, cache: maven }
      - run: ./mvnw -B verify          # unit + integration tests
  image:
    needs: build-test
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    permissions: { contents: read, packages: write }
    steps:
      - uses: actions/checkout@v4
      - uses: docker/login-action@v3
        with: { registry: ghcr.io, username: "${{ github.actor }}",
              password: "${{ secrets.GITHUB_TOKEN }}" }
      - run: docker build -t ghcr.io/example/order-service:${{ github.sha }} .
      - run: docker push ghcr.io/example/order-service:${{ github.sha }}
```

Two jobs

PRs only build and test; main also publishes an image.

Traceable tag

The commit SHA links the image back to the source.

Least privilege

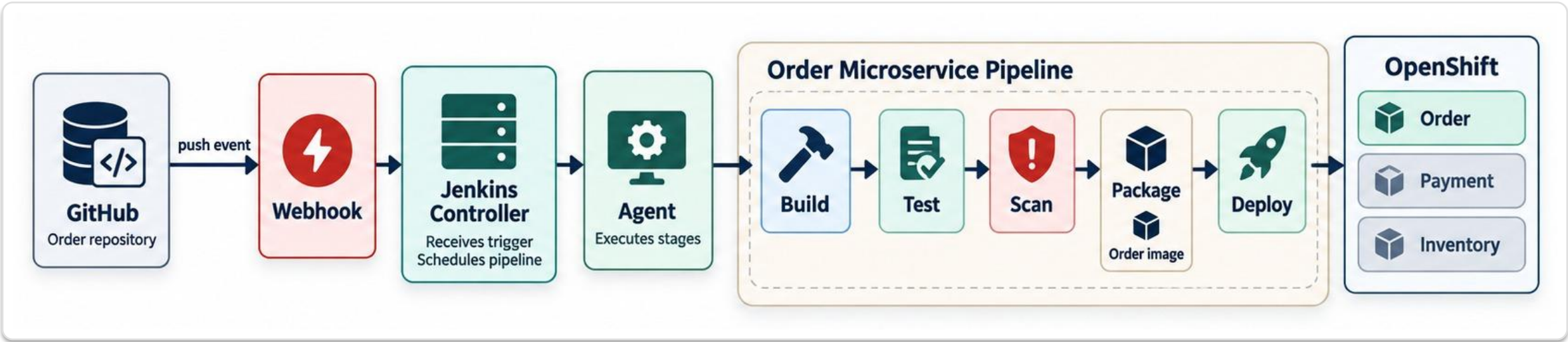
The job may read code and write packages — nothing more.

SECRETS From the secrets store — never typed into YAML.

KEY TAKEAWAY Keep the pipeline in the repository: test every PR, and publish a traceable image only from main.

JENKINS PIPELINE FOR MICROSERVICES

Controller and agents, Jenkinsfile, stages, artifacts and deployment.



Controller

Schedules pipelines, stores configuration and credentials.

Agents

Run the build steps, often inside the bank's network.

Jenkinsfile

Pipeline as code, reviewed in the same PR as the app.

IN THE LABS Jenkins is awareness only — nothing to install.

KEY TAKEAWAY A Jenkinsfile keeps delivery logic versioned and reviewed with the application.

A DECLARATIVE JENKINSFILE

Stages, test reports, stored credentials and a production approval.

✓ Jenkinsfile

```
pipeline {
  agent { label 'maven-jdk21' }
  environment { IMAGE = "registry.bank.local/orders/order-service:${GIT_COMMIT}" }
  stages {
    stage('Build & test') {
      steps { sh './mvnw -B verify' }
      post { always { junit 'target/surefire-reports/*.xml' } }
    }
    stage('Image') {
      when { branch 'main' }
      steps {
        withCredentials([string(credentialsId: 'registry-token', variable: 'TOKEN')]) {
          sh 'echo "$TOKEN" | podman login registry.bank.local -u ci --password-stdin'
          sh 'podman build -t "$IMAGE" . && podman push "$IMAGE"'
        }
      }
    }
    stage('Promote to prod') {
      when { branch 'main' }
      steps {
        input message: 'Promote this image to production?'
        sh './deploy.sh prod "$IMAGE"'
      }
    }
  }
}
```

Test reports

junit publishes results even when tests fail.

Credentials

withCredentials injects the token and masks it in logs.

Approval

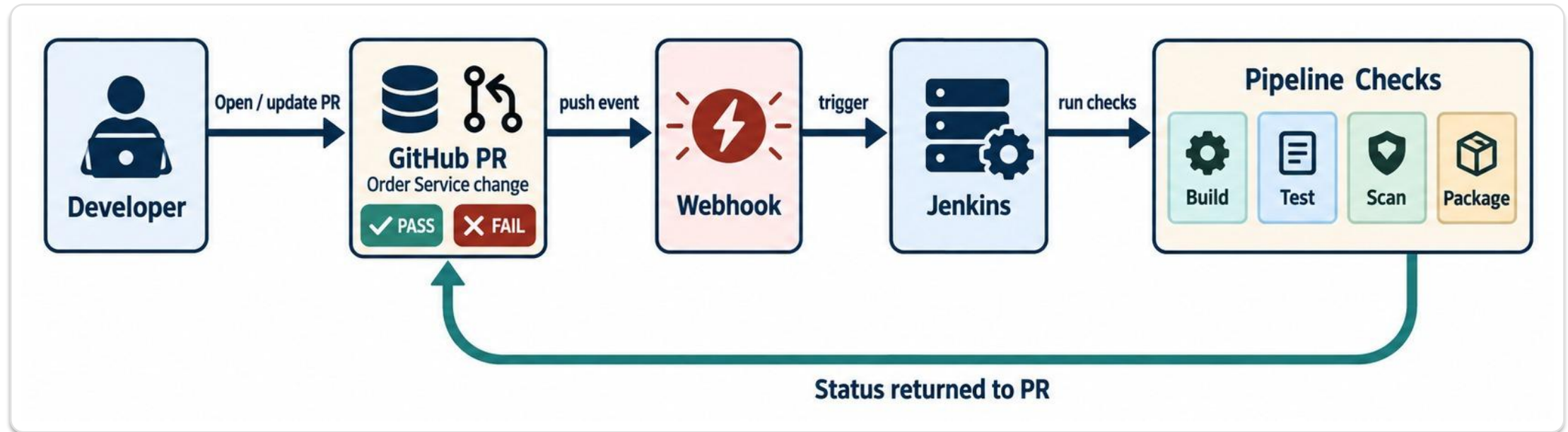
input pauses for a human — continuous delivery.

SAME IMAGE The image built once is the one promoted.

KEY TAKEAWAY A Jenkinsfile expresses the same pipeline — with stored credentials and an explicit production approval.

GITHUB AND JENKINS TOGETHER

A GitHub PR triggers a Jenkins pipeline, which reports status back.



GitHub

Repository, pull requests, reviews and branch protection.

Jenkins

Runs the pipeline on internal agents with internal credentials.

The link

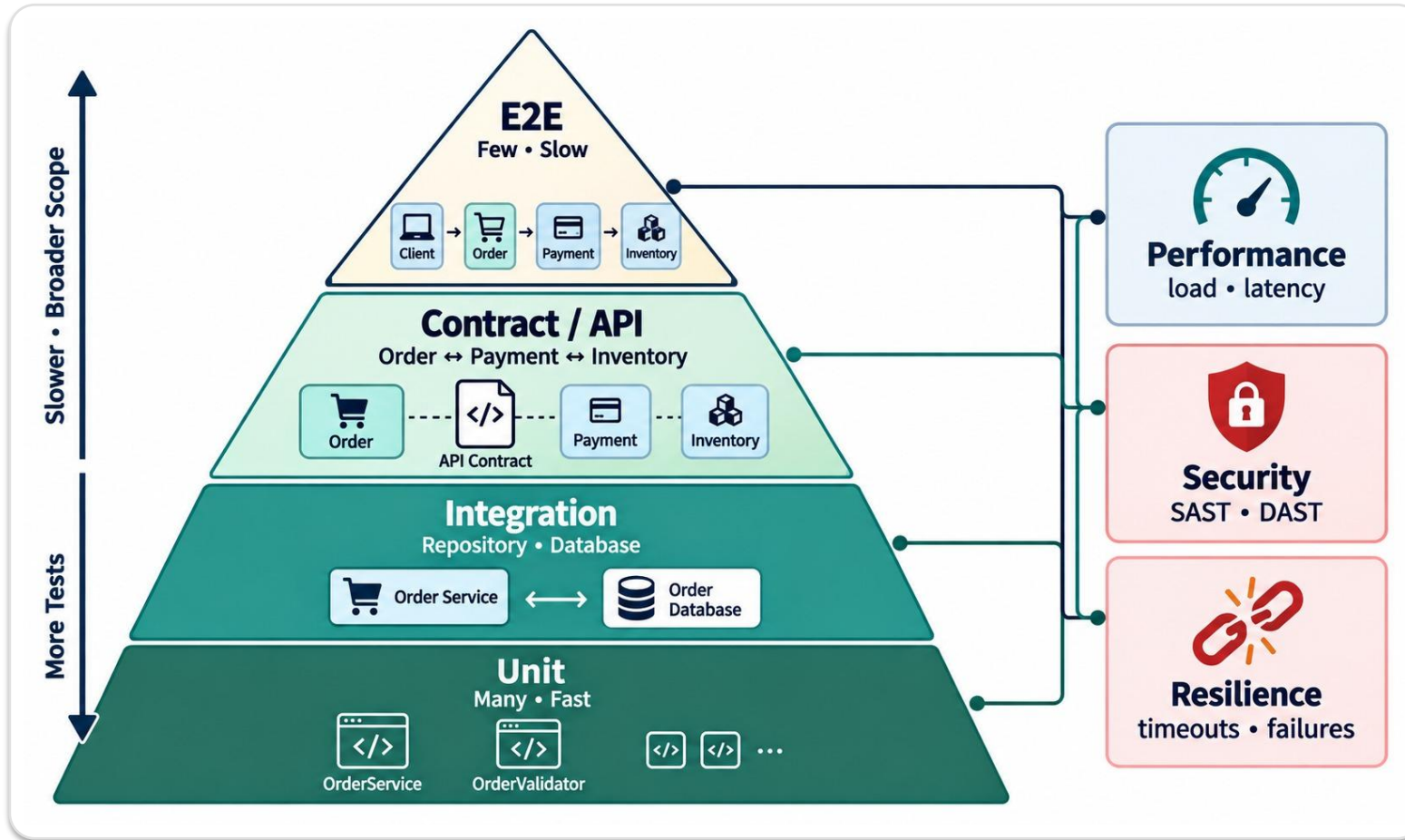
A webhook starts the build; a commit status marks the PR green or red.

RESULT Reviewers see pipeline results right in the PR.

KEY TAKEAWAY GitHub for collaboration, Jenkins for execution — they work together.

AUTOMATED TESTING STRATEGY

Unit, integration, contract, API, end-to-end, performance and security.



EACH LAYER CATCHES

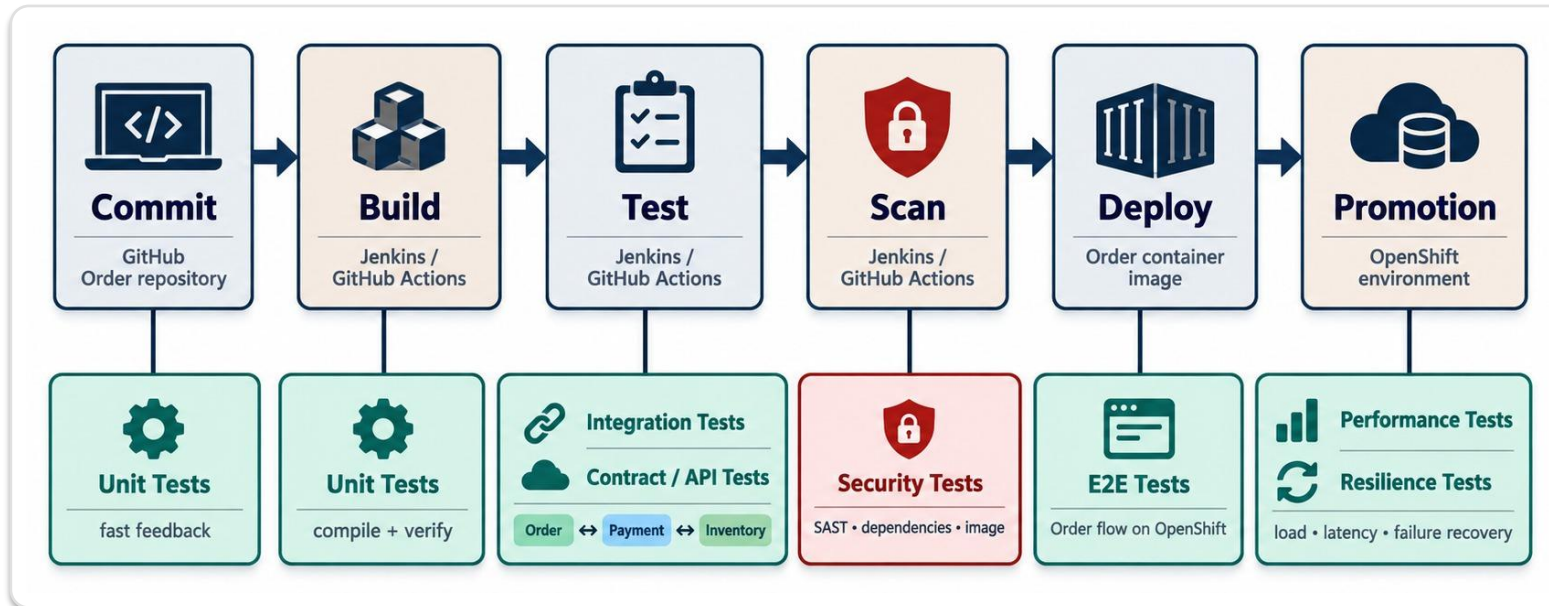
- ✓ Unit: logic bugs
- ✓ Integration: wiring bugs
- ✓ Contract: compatibility bugs
- ✓ Smoke: deployment bugs
- ✓ Security: dangerous exposure

AVOID one giant, slow, flaky end-to-end suite.

KEY TAKEAWAY Don't rely on one giant end-to-end suite — layer your tests.

TESTING IN THE CI/CD PIPELINE

Map each test type to the right pipeline stage.



On every PR

Unit, slice and contract tests — minutes.

After merge

Integration with Testcontainers, image and security scans.

After deploy

Smoke, API and performance checks before promotion.

PRINCIPLE Fail early and cheaply; spend minutes, not hours.

KEY TAKEAWAY Run cheap tests first and slower tests later — balance speed, coverage and confidence.

TESTS THE PIPELINE RUNS

A security slice test and a real-database integration test — every build.

✓ AccountSecurityTest.java

```
@WebMvcTest(AccountController.class)
@Import(SecurityConfig.class)
class AccountSecurityTest {
    @Autowired MockMvc mvc;
    @MockitoBean AccountService accounts;

    @Test void noTokenIs401() throws Exception {
        mvc.perform(get("/api/v1/accounts/ACC-1"))
            .andExpect(status().isUnauthorized());
    }

    @Test void wrongScopeIs403() throws Exception {
        mvc.perform(get("/api/v1/accounts/ACC-1")
            .with(jwt().authorities(
                new SimpleGrantedAuthority("SCOPE_transactions.read"))))
            .andExpect(status().isForbidden());
    }
}
```

Security slice

Proves 401 without a token and 403 with the wrong scope.

AccountPersistenceTest.java

```
@DataJpaTest
@Testcontainers
@AutoConfigureTestDatabase(replace = NONE)
class AccountPersistenceTest {
    @Container @ServiceConnection
    static PostgreSQLContainer<?> db =
        new PostgreSQLContainer<>("postgres:16");

    @Autowired AccountRepository repo;

    @Test void savesAndReadsAccount() {
        repo.save(Account.pending("CUST-0001"));
        assertThat(repo.findAll()).hasSize(1);
    }
}
```

Testcontainers

A real Postgres 16 — Flyway migrations run for real.

In CI

Runners need Docker; both tests gate the merge.

KEY TAKEAWAY Automate the tests that protect security rules and real persistence — and let them block the merge.

EXERCISE 3.3 — ADD SECURITY AND RESILIENCE TESTS

Day 3 checkpoint C: unit tests plus one integration test.

OFFICIAL CHECKPOINT C

10 MIN

labs/day-03/exercises/exercise-3.3-security-resilience-tests.md

SCENARIO

Lab 3 secured and hardened the banking Account and Transaction services. Now the pipeline must prove those rules on every build.

Scope: Account, Transaction and Risk Assessment only.

YOUR TASKS

- 1 Unit-test 401 and 403 on a protected Account or Transaction endpoint
- 2 Unit-test circuit open → 503 fallback — no fake ACTIVE account
- 3 Name AccountPersistenceTest — Testcontainers Postgres 16; Docker must run for mvn test
- 4 Testcontainers is required for that integration test, not optional

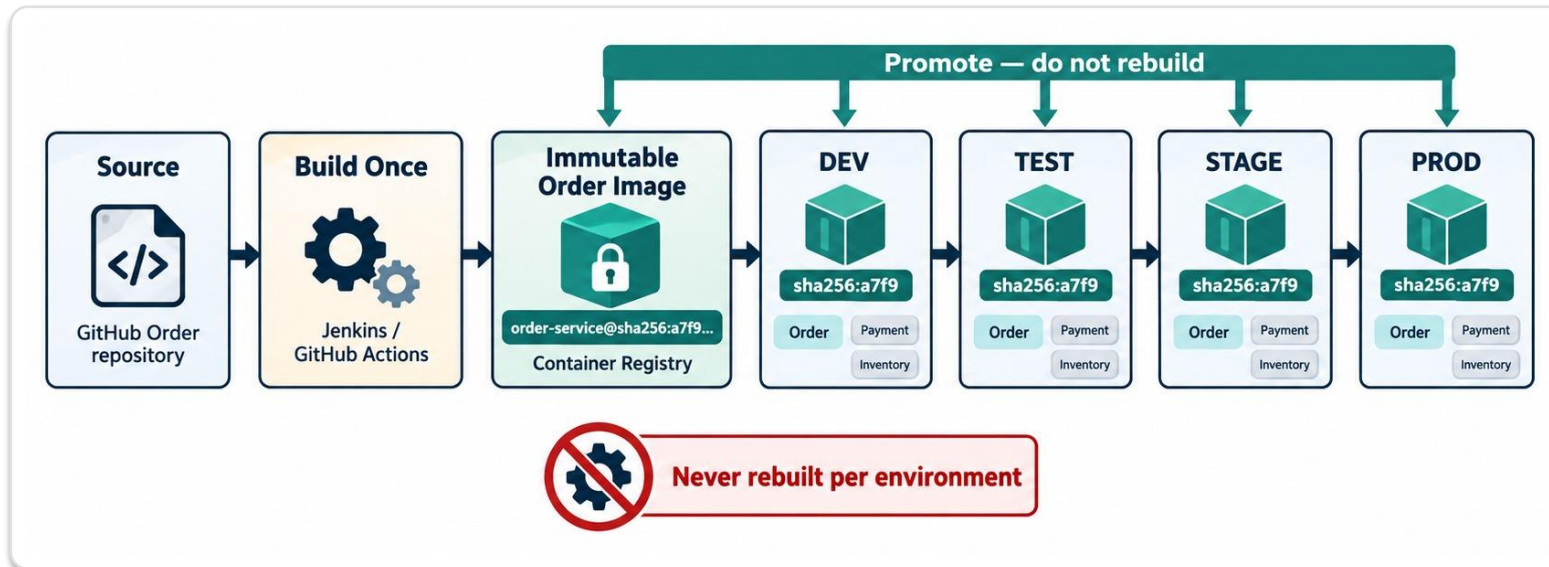
EXPECTED OUTCOME

- ✓ A short test list Lab 3 can tick
- ✓ Security rules covered
- ✓ The fallback rule covered
- ✓ One real-database integration test

KEY TAKEAWAY A short test list that proves security and the fallback rule on every build.

BUILD ONCE, PROMOTE THE SAME ARTIFACT

One immutable artifact promoted through DEV → TEST → STAGE → PROD.



THE DANGEROUS MODEL

- ✗ Rebuild per environment
- ✗ Different dependencies each time
- ✗ “Worked in test” means nothing

What changes between environments

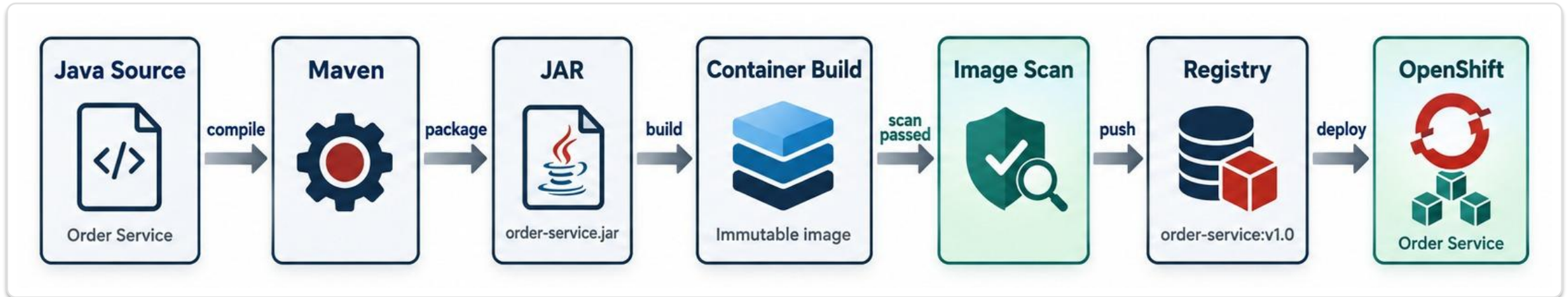
Only ConfigMaps, Secrets and replica counts — never the image.

PROMOTE BY DIGEST sha256:... guarantees the exact same bytes.

KEY TAKEAWAY What we tested is what we deploy — same artifact, different external configuration.

CONTAINER IMAGE AND REGISTRY FLOW

Source → build image → tag → scan → registry → deployment.



✓ A traceable tag

```
registry.bank.local/orders/order-  
service:1.4.0-a13f92c
```

AVOID

✗ latest alone — which build is this?

Traceability

Tag → pipeline run → commit →
pull request → work item.

MODULE 8 goes deeper into
images and registries.

KEY TAKEAWAY Versioned, scanned images in a registry make every production release traceable.

PROMOTE THE SAME IMAGE

Tag once in CI, then move the same digest from environment to environment.

CI — build and record the digest

```
VERSION=1.4.0
SHA=$(git rev-parse --short HEAD)
IMAGE=registry.bank.local/orders/order-service

podman build -t $IMAGE:$VERSION-$SHA .
podman push --digestfile digest.txt $IMAGE:$VERSION-$SHA
# digest.txt → sha256:9f2c... (store with the run)
```

Version + SHA

Readable for humans, traceable to the commit.

Digest

A tag can be moved; a digest cannot.

Config outside

Each namespace has its own ConfigMap and Secret.

✓ Promotion — no rebuild

```
DIGEST=$(cat digest.txt)

# TEST
oc -n orders-test set image deployment/order-service \
  order-service=$IMAGE@$DIGEST
oc -n orders-test rollout status deployment/order-service

# PROD — after approval, the very same digest
oc -n orders-prod set image deployment/order-service \
  order-service=$IMAGE@$DIGEST
```

KEY TAKEAWAY Build and tag once, then promote the exact digest — never rebuild for production.

EXERCISE 4.3 — TRACE A PIPELINE STAGE OWNERSHIP MAP

Day 4 checkpoint C: who owns scan, SBOM, sign and promote.

OFFICIAL CHECKPOINT C

15 MIN

labs/day-04/lab4/starter/pipeline/OWNERSHIP.md

SCENARIO

Lab 4 walks a prepared pipeline for the Account and Transaction services: checkout, compile, test, scan, image, SBOM, sign, deploy, smoke and approval.

YOUR TASKS

- 1 Open OWNERSHIP.md and fill who owns scan, SBOM, sign and promote
- 2 A CRITICAL CVE fails the gate; the SBOM is the ingredients list; unsigned images must not run
- 3 Participants execute and verify — they don't manage signing keys
- 4 GitOps is conceptual — don't install Jenkins or GitHub Actions runners

EXPECTED OUTCOME

- ✓ A completed ownership map
- ✓ Each stage's failure meaning understood
- ✓ Compare with the solution OWNERSHIP.md
- ✓ Fill it before run-pipeline-locally.ps1

KEY TAKEAWAY Every pipeline stage has an owner and a clear meaning when it fails.

PART 3: RELEASE STRATEGIES

Deploying is not the same as exposing every user to the change.



SO FAR

- ✓ CI proves each change; CD keeps it releasable
- ✓ Tests run at the right pipeline stage
- ✓ One image is promoted everywhere

NEXT QUESTION

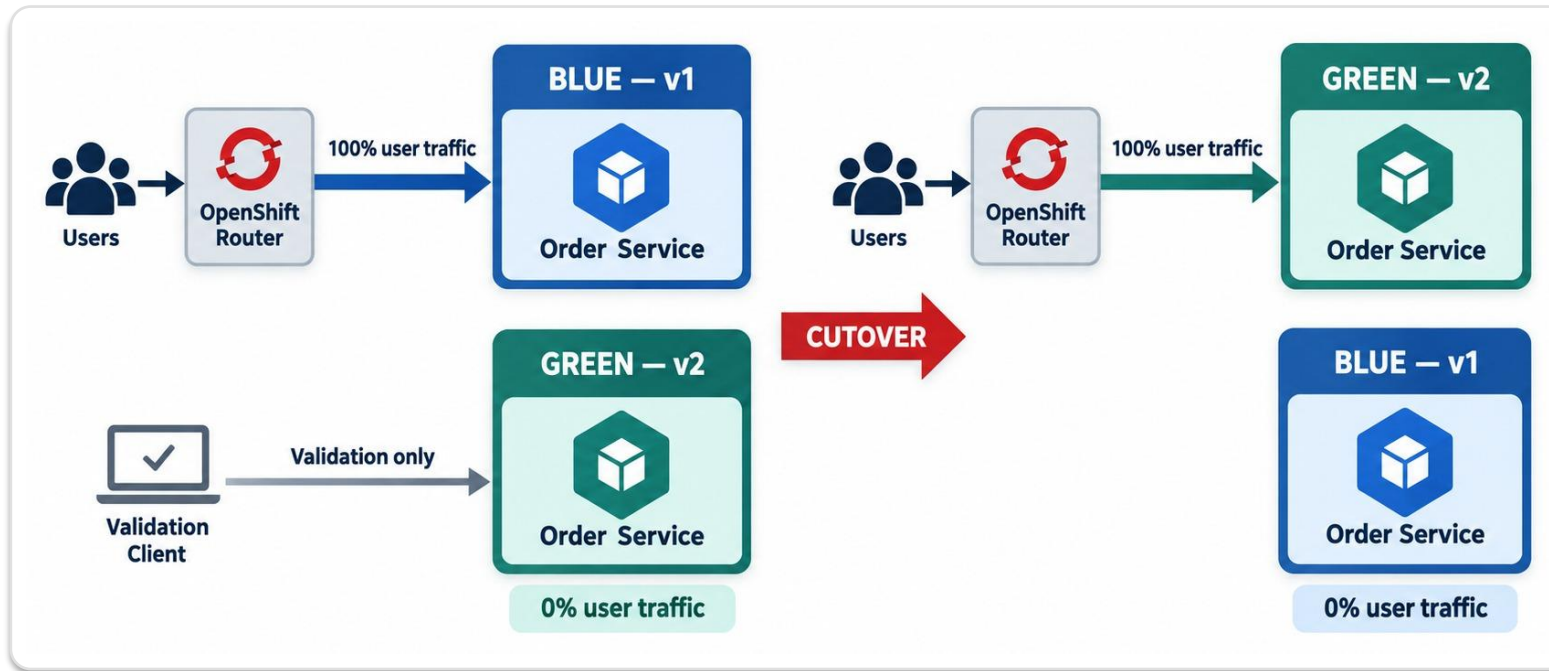
How do we release a new version without risking every customer at once?

IN THIS PART

- 1 Blue/Green deployment
- 2 Canary deployment
- 3 Choosing between them
- 4 Traffic switching on OpenShift — in code
- 5 Exercise: choose a release strategy

BLUE/GREEN DEPLOYMENT

Two production environments — traffic switched between Blue and Green.



ADVANTAGES

- ✓ Instant cutover
- ✓ Instant rollback
- ✓ Test in production before exposure

TRADE-OFFS

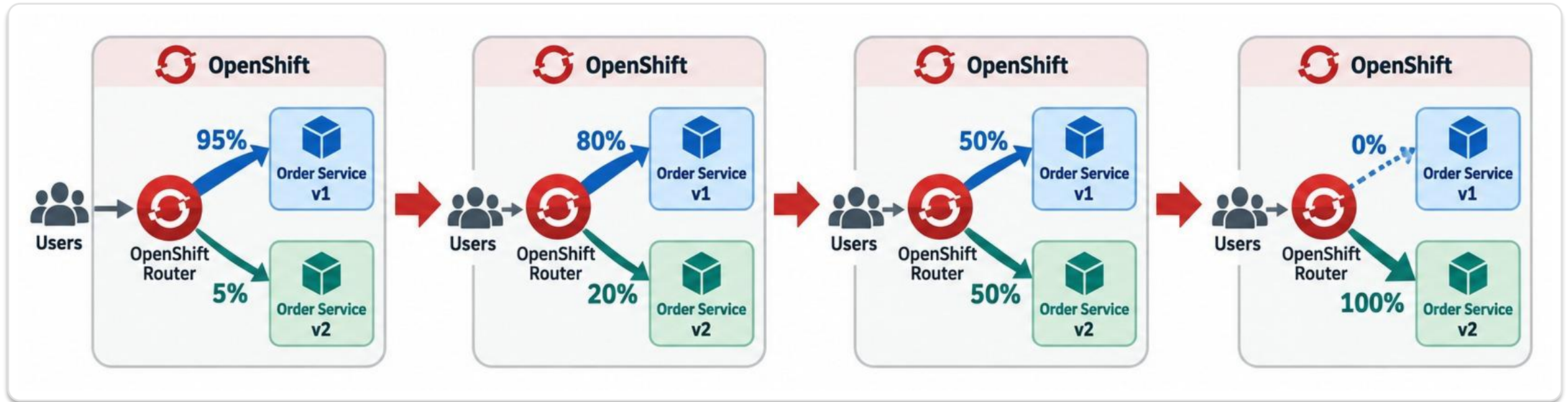
- ! Double capacity during release
- ! Schema must suit both versions

DATA Use expand-and-contract migrations.

KEY TAKEAWAY Blue/Green switches all traffic at once — and can switch it back quickly.

CANARY DEPLOYMENT

Gradually shift a small percentage of traffic to the new version.



WATCH AT EACH STEP

Error rate

p95 latency

Failed payments

Saturation

Automatic rollback

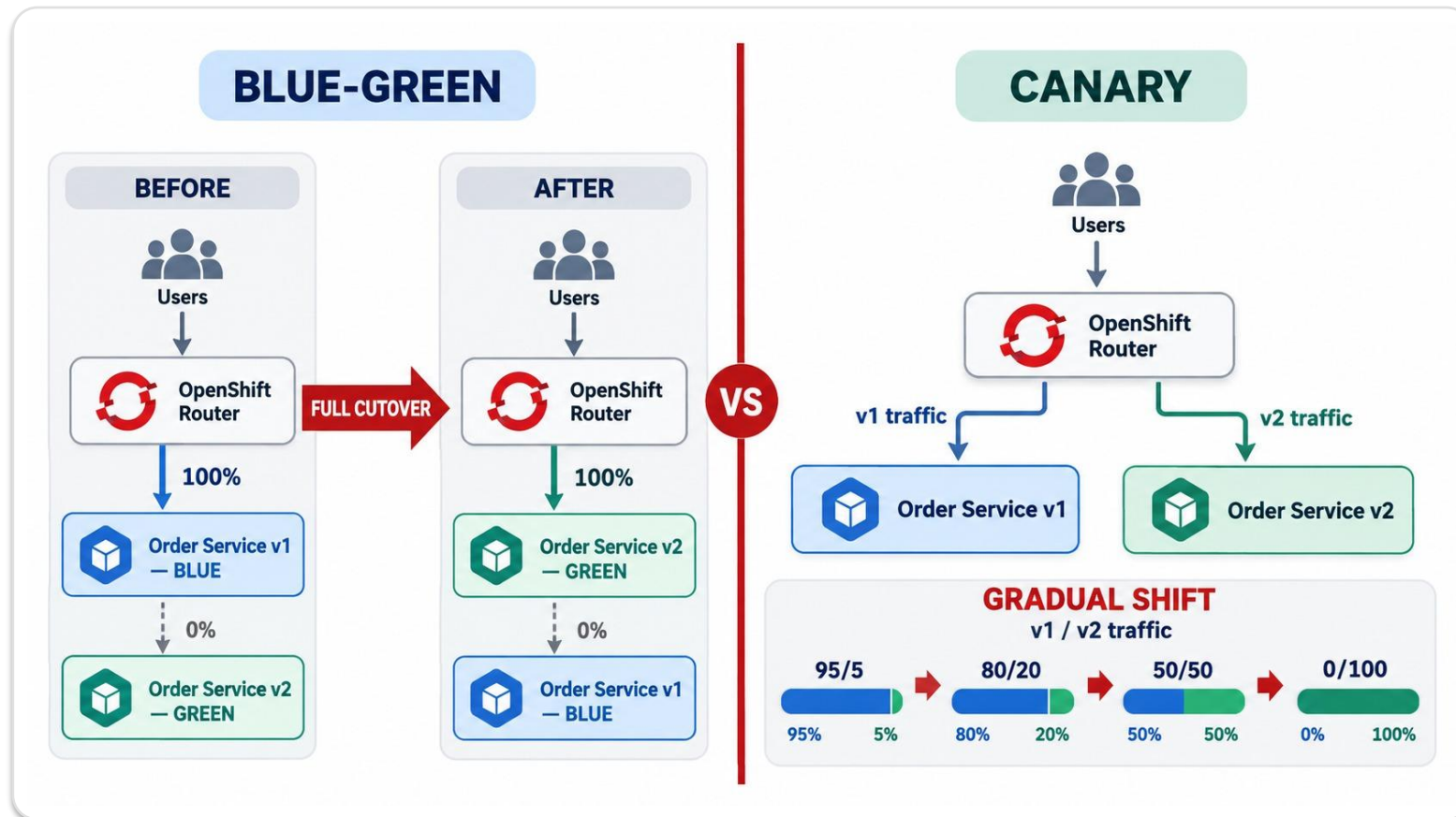
If a signal crosses its threshold, return all traffic to v1.

NEEDS good observability — Module 6.

KEY TAKEAWAY Canary exposes a new release gradually — and depends on good observability.

BLUE/GREEN VS. CANARY DEPLOYMENT

Risk, rollback, infrastructure cost, traffic control and use cases.



Choose Blue/Green

Fast, all-or-nothing releases with an instant rollback path.

Choose canary

Risky changes where real-traffic evidence should drive exposure.

Consider

Infrastructure cost, data compatibility and how good your metrics are.

BOTH need automated health checks.

KEY TAKEAWAY Blue/Green changes where all traffic goes; Canary changes how much goes to the new release.

SWITCHING TRAFFIC WITH OPENSIFT ROUTES

The same Route can cut over all traffic — or split it by weight.

Blue/Green — switch the backend

```
apiVersion: route.openshift.io/v1
kind: Route
metadata:
  name: order-service
spec:
  to:
    kind: Service
    name: order-service-green # was -blue
  tls:
    termination: edge

# roll back:
# oc patch route order-service \
#   -p '{"spec":{"to":{"name":"order-service-blue"}}}'
```

Blue/Green

Change one field; change it back to roll back.

✓ Canary — weighted backends

```
apiVersion: route.openshift.io/v1
kind: Route
metadata:
  name: order-service
spec:
  to:
    kind: Service
    name: order-service-v1
    weight: 95
  alternateBackends:
    - kind: Service
      name: order-service-v2
      weight: 5
  tls:
    termination: edge
```

Canary

Raise v2's weight step by step: 5, 20, 50, 100.

Automate it

Tools such as Argo Rollouts drive the steps from metrics.

KEY TAKEAWAY Routes give both strategies: repoint the whole backend, or shift traffic by weight.

EXERCISE: CHOOSE A RELEASE STRATEGY

Pick Blue/Green, canary or a rolling update for each release.

PRACTICE EXERCISE

10 MIN

SCENARIO

Four releases are planned this month:

- A Payment Service: new fraud-check call
- B Order Service: fix a typo in an error message
- C Inventory Service: rename a DB column
- D Transfer API: new routing to a partner bank

YOUR TASKS

- 1 Choose a strategy for each release
- 2 Name the signals that would stop the rollout
- 3 Describe the rollback — including data

EXPECTED OUTCOME

- ✓ A: canary — watch failed payments and latency
- ✓ B: rolling update — low risk
- ✓ C: expand-and-contract first, then Blue/Green
- ✓ D: canary to a small share, with approval
- ✓ Rollback: repoint the Route; schema stays compatible

KEY TAKEAWAY Match the release strategy to the risk — and plan the data rollback before the traffic rollback.

PART 4: SECURITY IN THE PIPELINE

Whoever controls the pipeline can change production.



SO FAR

- ✓ Blue/Green cuts over and back fast
- ✓ Canary grows exposure on healthy signals
- ✓ Routes switch or split traffic

NEXT QUESTION

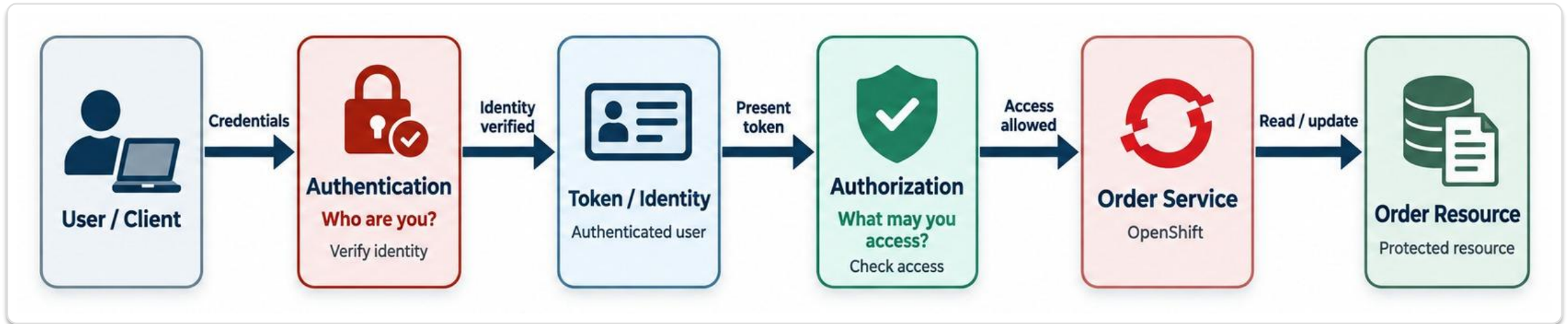
How do we make sure only trusted, secure changes reach production?

IN THIS PART

- 1 Authentication and authorization basics
- 2 DevSecOps scanning and policy gates
- 3 Securing a workflow — in code
- 4 Exercise: find the pipeline security problems
- 5 The complete delivery architecture

AUTHENTICATION AND SECURITY BASICS

Identity, authentication, authorization, tokens, secrets and least privilege.



For people

SSO and MFA for Git, CI and cluster access.

For pipelines

Short-lived tokens with only the permissions each job needs.

For services

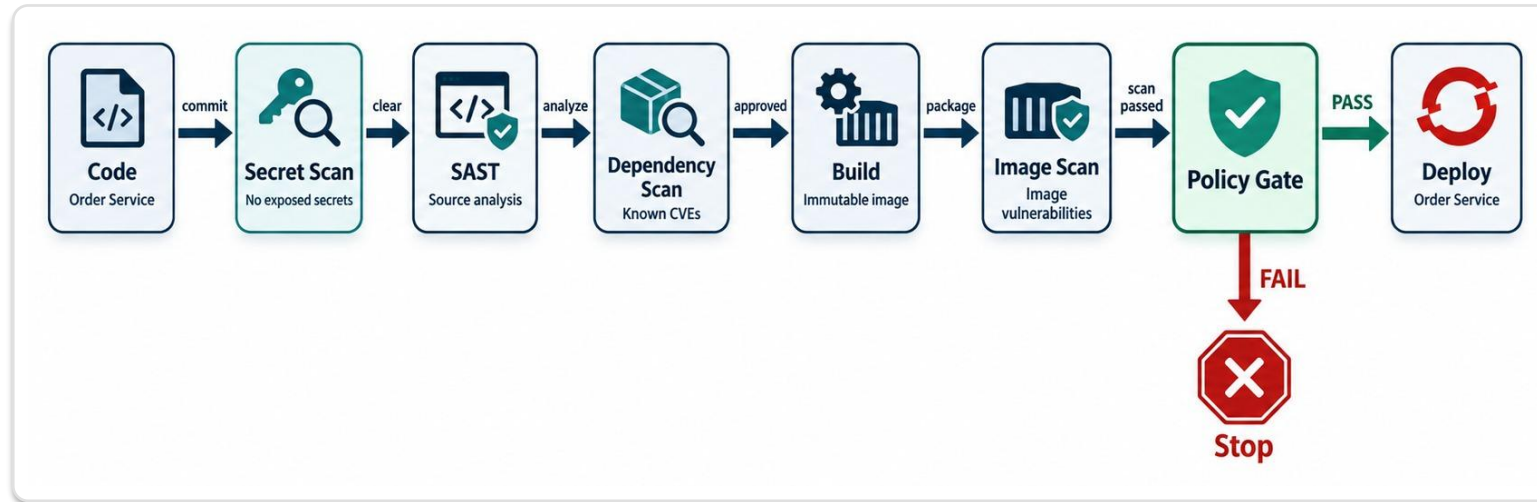
OAuth2 tokens or workload identity — not network trust.

MODULE 12 covers OAuth2, OIDC and JWT in depth.

KEY TAKEAWAY Authentication proves who you are; authorization decides what you may do.

DEVSECOPS IN THE PIPELINE

SAST, dependency scanning, secrets scanning, image scanning and policy gates.



TYPICAL TOOLS

Gitleaks

SonarQube

Dependency-Check

Trivy

Syft (SBOM)

Cosign

Policy gate

No CRITICAL CVEs, an SBOM attached and a valid signature — or no deploy.

SHIFT LEFT A leaked secret found in the PR is cheap to fix.

KEY TAKEAWAY Security becomes part of the normal engineering and delivery workflow.

SECURITY SCANS IN A WORKFLOW

Secrets, dependencies and the image are scanned — and CRITICAL findings stop the run.

✓ .github/workflows/security.yml (excerpt)

```
jobs:
  scan:
    runs-on: ubuntu-latest
    permissions: { contents: read }
    steps:
      - uses: actions/checkout@v4
        with: { fetch-depth: 0 }

      - name: Secret scan
        uses: gitleaks/gitleaks-action@v2

      - name: Dependency and image scan
        uses: aquasecurity/trivy-action@0.28.0
        with:
          image-ref: registry.bank.local/orders/order-service:${{ github.sha }}
          severity: CRITICAL
          exit-code: '1'           # fail the job

      - name: SBOM
        run: syft registry.bank.local/orders/order-service:${{ github.sha }} \
          -o spdx-json > sbom.spdx.json
```

Secret scan

Blocks the PR if a key or password was committed.

Gate

exit-code 1 fails the job on any CRITICAL finding.

SBOM

The ingredients list you need when a new CVE appears.

PIN VERSIONS Pin actions to a version or commit.

KEY TAKEAWAY Scan secrets, dependencies and images in every run — and let CRITICAL findings block the release.

EXERCISE: FIND THE PIPELINE SECURITY PROBLEMS

Review a pipeline change before it merges.

PRACTICE EXERCISE

12 MIN

SCENARIO

A teammate proposes this deploy job:

```
deploy:
  runs-on: ubuntu-latest
  permissions: write-all
  env:
    DB_PASSWORD: SuperSecret123
  steps:
    - uses: some-user/deploy-action@main
    - run: docker build -t order:latest .
    - run: echo $DB_PASSWORD
    - run: ./deploy.sh prod order:latest
```

YOUR TASKS

- 1 Find at least five problems
- 2 Explain the risk of each
- 3 Rewrite the job safely

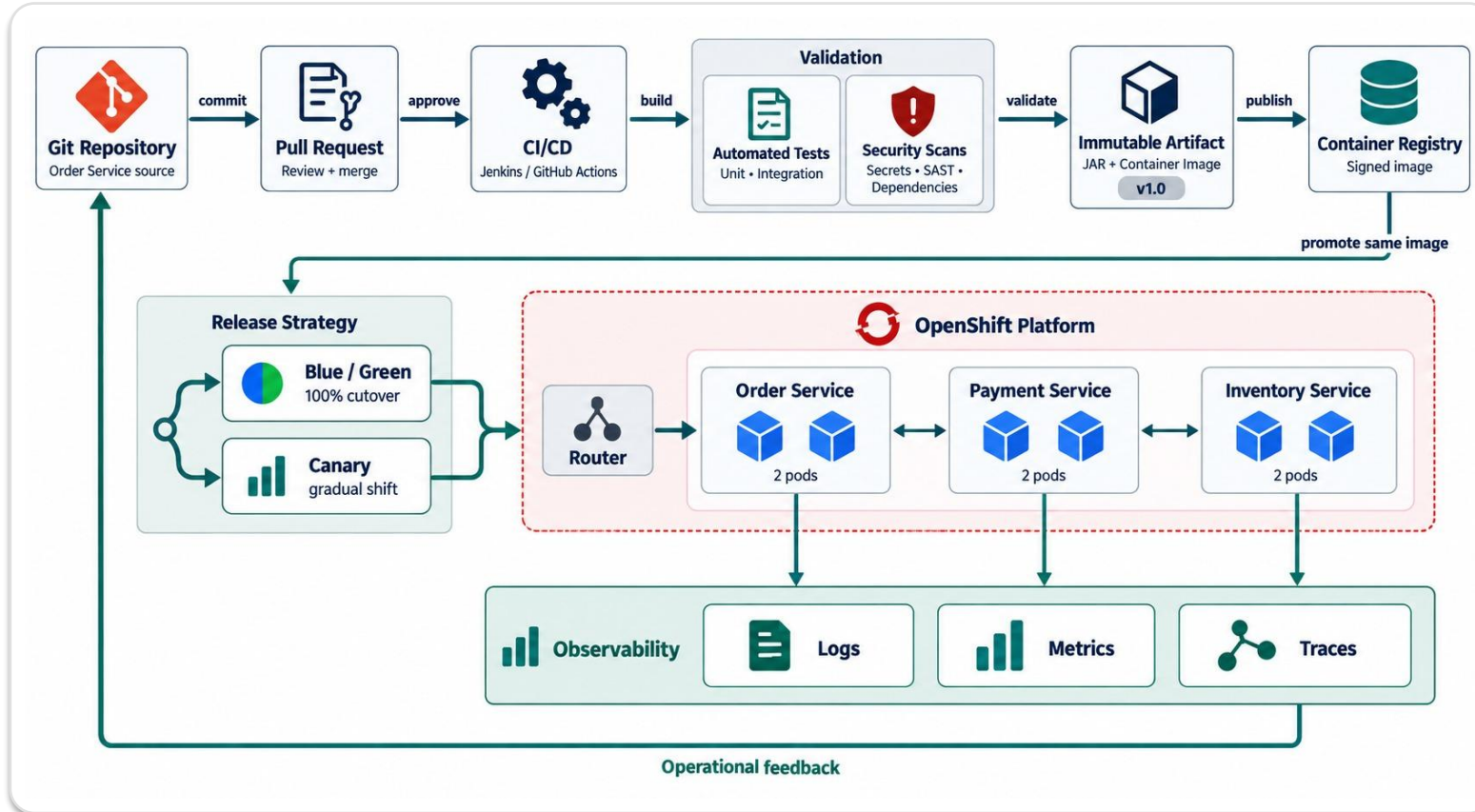
EXPECTED OUTCOME

- ✓ write-all → least-privilege permissions
- ✓ Password in YAML → secrets store
- ✓ Unpinned third-party action → pin to a commit
- ✓ latest, rebuilt → promote the scanned digest
- ✓ Echoing a secret; no scan or approval gate

KEY TAKEAWAY Treat pipeline code like production code: least privilege, no secrets, pinned dependencies and gates.

END-TO-END MICROSERVICES DEVOPS DELIVERY ARCHITECTURE

Git → PR → CI/CD → tests and security → registry → deployment strategy → observability.



EVERY PART OF THE MODULE

- ✓ Git and pull requests gate changes
- ✓ CI tests and scans every build
- ✓ One signed image is promoted
- ✓ Blue/Green or canary on OpenShift
- ✓ Logs, metrics and traces feed back

REMEMBER What we tested is what we deploy.

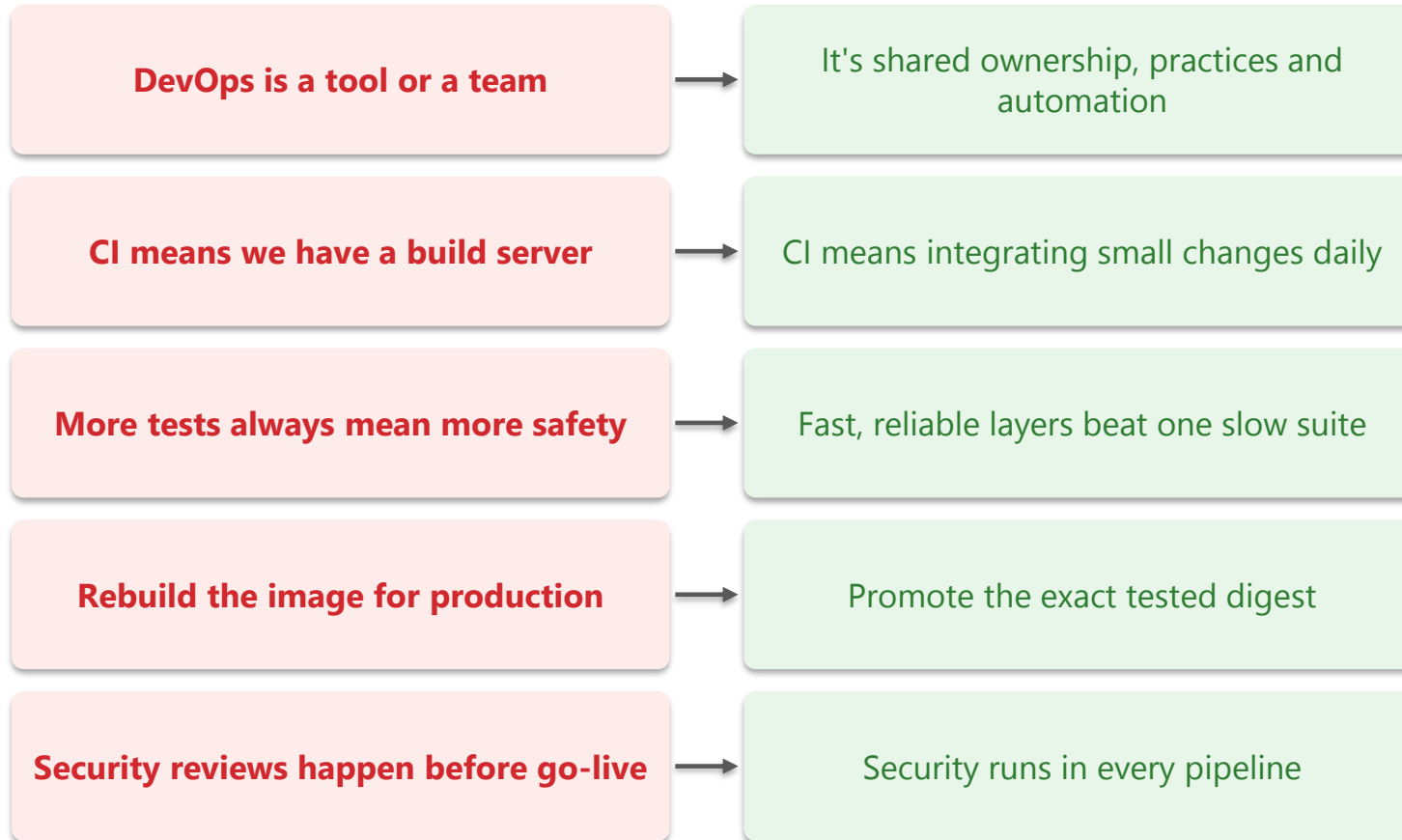
KEY TAKEAWAY DevOps turns microservice delivery into a repeatable, measurable feedback loop.

DEVOPS MISCONCEPTIONS AND ANTI-PATTERNS

Beliefs and habits that make delivery slow, fragile or unsafe.

MISCONCEPTION

REALITY



ANTI-PATTERNS TO AVOID

- ✗ Long-lived feature branches
- ✗ Manual, undocumented deployments
- ✗ Deploying latest
- ✗ Secrets in pipeline YAML
- ✗ Skipping tests to meet a deadline
- ✗ No rollback plan

REMEMBER Pipeline = evidence. Main = always releasable.

KEY TAKEAWAY Good delivery is small changes, automated evidence and a rehearsed rollback — every time.

KNOWLEDGE CHECK

Answer in your own words — then compare with a partner.

1 What is the purpose of DevOps?

2 Why are short-lived feature branches useful?

3 What is a pull request for?

4 What is a Jenkinsfile?

5 What does a CI pipeline validate?

6 Continuous delivery vs. continuous deployment?

7 Why run automated tests before merge?

8 What does static code analysis detect?

9 How does Blue/Green support rollback?

10 How does canary deployment reduce risk?

MODULE 7 SUMMARY AND WHAT'S NEXT

From a reviewed commit to a safe, observable production release.

YOU CAN NOW

- ✓ Run a Git workflow with pull-request quality gates
- ✓ Explain CI, continuous delivery and continuous deployment
- ✓ Read a GitHub Actions workflow and a Jenkinsfile
- ✓ Build once and promote the same image
- ✓ Choose Blue/Green or canary and gate on security scans

NEXT: MODULE 8 — CONTAINERIZATION WITH OPENSIFT

- 1 What exactly is inside a container image?
- 2 How does OpenShift run, expose and scale a service?
- 3 How do probes, config and rollouts work on the platform?

BRING ALONG Your Lab 3 services — Lab 4 containerizes and deploys them.

KEY TAKEAWAY Module 8 looks inside the images this pipeline builds — and the platform that runs them.